CPSC 250
Recursion, Binary Search, and Algorithm Efficiency

Edward Brash

1 Recursion

A recursive function is a function that calls itself.

Recursion can be useful when problems naturally break into smaller versions of themselves.

2 A Bad Recursive Example

Consider:

```
def count_down(count):

    count_down(count - 1)
```

What happens if we call:

```
count_down(3)
```

The function repeatedly calls itself forever:

$$3 \to 2 \to 1 \to 0 \to -1 \to \cdots$$

Eventually Python crashes with a recursion error.

3 The Importance of an End Condition

Recursive functions require an end condition.

```
def count_down(count):

    if count == 0:
        print("Go!")

    else:
        print(count)
        count_down(count - 1)
```

Now:

$$count\_down(3)$$

produces:

$$3, 2, 1, Go!$$

4 Recursion vs Iteration

Although recursion can be elegant, it is often:

- harder to understand,
- harder to debug,
- not more efficient.

One major exception is binary-search-style algorithms.

5 Binary Search Motivation

Suppose we want to find the root of a function:

$$f(x) = 0$$

Assume we know the root lies somewhere between:

$$x_{low}$$

and:

$$x_{high}$$

6 Root Condition

A root exists between two points if:

$$f(x_{low}) \cdot f(x_{high}) < 0$$

This means the function changes sign.

7 The Binary Search Idea

Suppose the root lies between:

$$a$$

and:

$$b$$

We compute the midpoint:

$$c = \frac{a + b}{2}$$

Then determine which half contains the root.

8 Binary Search Algorithm

1. Compute midpoint:

$$c = \frac{a+b}{2}$$

2. Evaluate:

$$f(a)f(c)$$

3. If:

$$f(a)f(c) < 0$$

then the root lies between a and c.

Otherwise it lies between c and b.

4. Repeat.

9 Stopping Condition

The algorithm ends when the interval becomes sufficiently small:

$$|b-a| < \epsilon$$

where:

$$\epsilon$$

is a tiny number.

10 Example Function

```
def f(x):
    return -4*x + 3
```

This function has a root at:

$$x = 0.75$$

11 Starting Interval

```
a = 0.0
b = 5.0
```

Check:

```
f(a) * f(b)
```

If the product is negative, a root exists on the interval.

12 Recursive Binary Search Root Finder

```
def find_root(a, b):

    epsilon = 1.0E-5

    if abs(b - a) < epsilon:
        print("Found root at x =", a)

    else:

        c = (a + b) / 2.0

        if f(a) * f(c) < 0:
            find_root(a, c)

        else:
            find_root(c, b)
```

13 Searching Sorted Data

Binary search is also used to search sorted lists.

Suppose we have alphabetically sorted names:

Alice
Charlie
Dave
Frieda
Kallie
Moe
Paul
Sarah
Xavier

We repeatedly check the middle item and eliminate half of the remaining list.

14 Algorithm Efficiency

Algorithm efficiency describes how runtime scales with input size.

This is usually expressed using Big-O notation.

Common examples:

$$O(n)$$

$$O(n^2)$$

$$O(\log n)$$

15 Linear Search

Searching a list one item at a time requires approximately:

$$\frac{n}{2}$$

checks on average.
Complexity:

$$O(n)$$

16 Binary Search Efficiency

Binary search repeatedly cuts the search space in half.
If:

$$n = 2^h$$

then:

$$h = \log_2(n)$$

Therefore binary search complexity is:

$$O(\log n)$$

17 Why Logarithmic Complexity Is Powerful

Suppose:

$$n = 100,000,000$$

A linear algorithm may require roughly:

$$100,000,000$$

operations.
But binary search only requires approximately:

$$\log_2(100,000,000) \approx 27$$

steps.
This is dramatically faster.

18 Key Takeaways

- Recursive functions call themselves.
- Recursive algorithms require stopping conditions.
- Binary search repeatedly divides a problem in half.

- Binary search is extremely efficient.
- Big-O notation describes scaling behavior.
- Logarithmic algorithms are vastly faster than linear algorithms for large datasets.